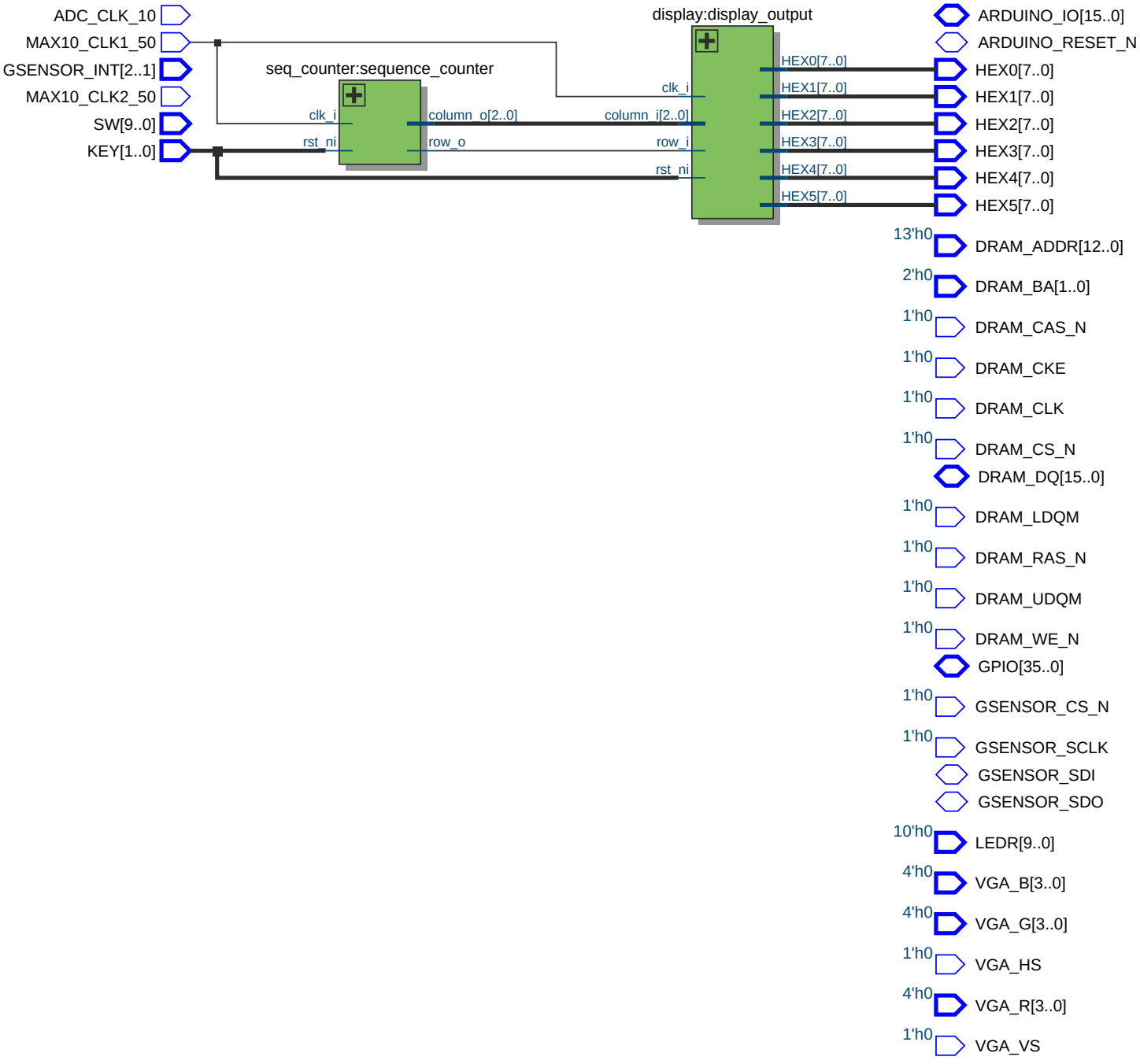
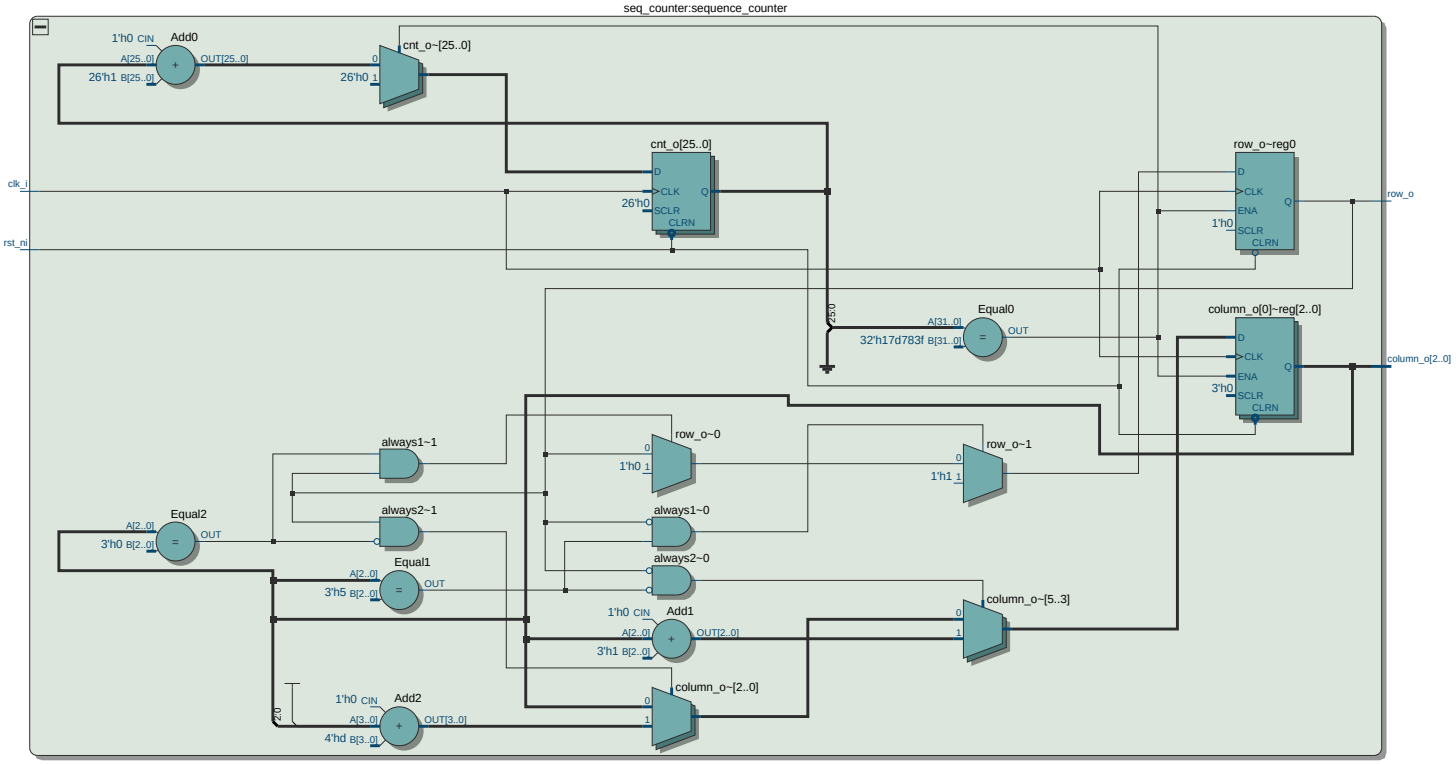
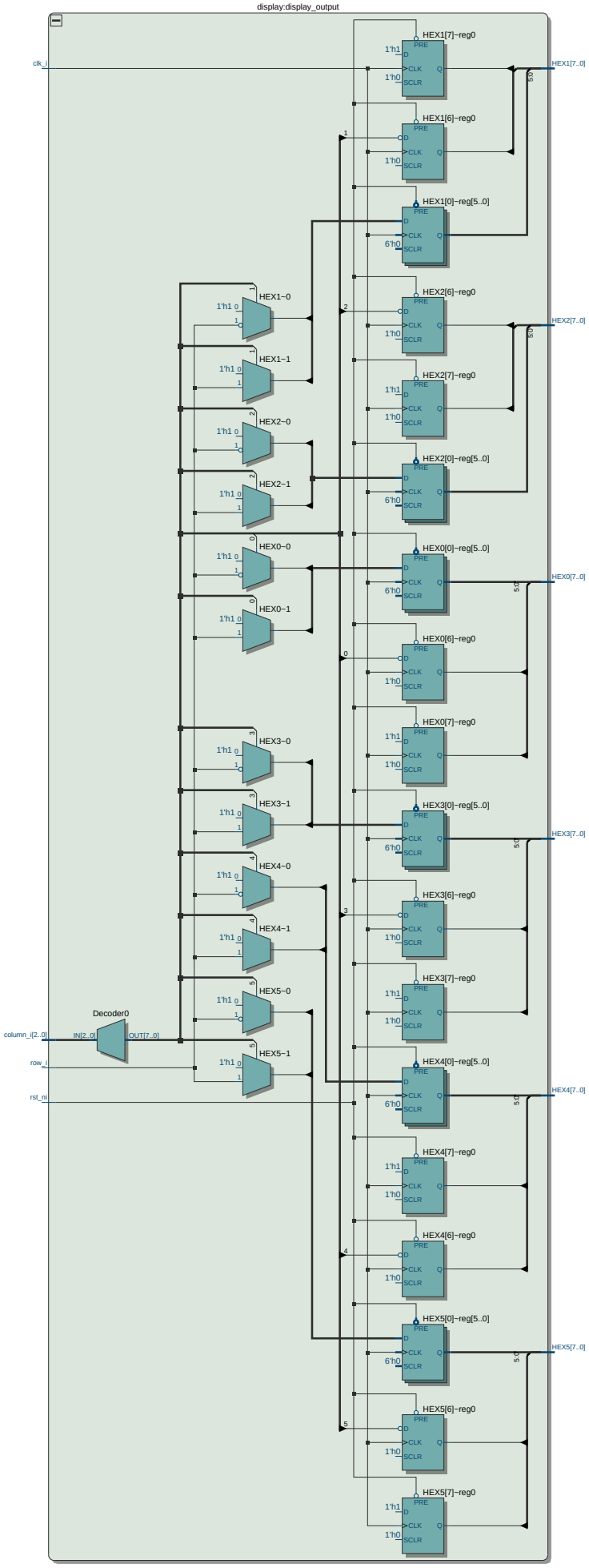
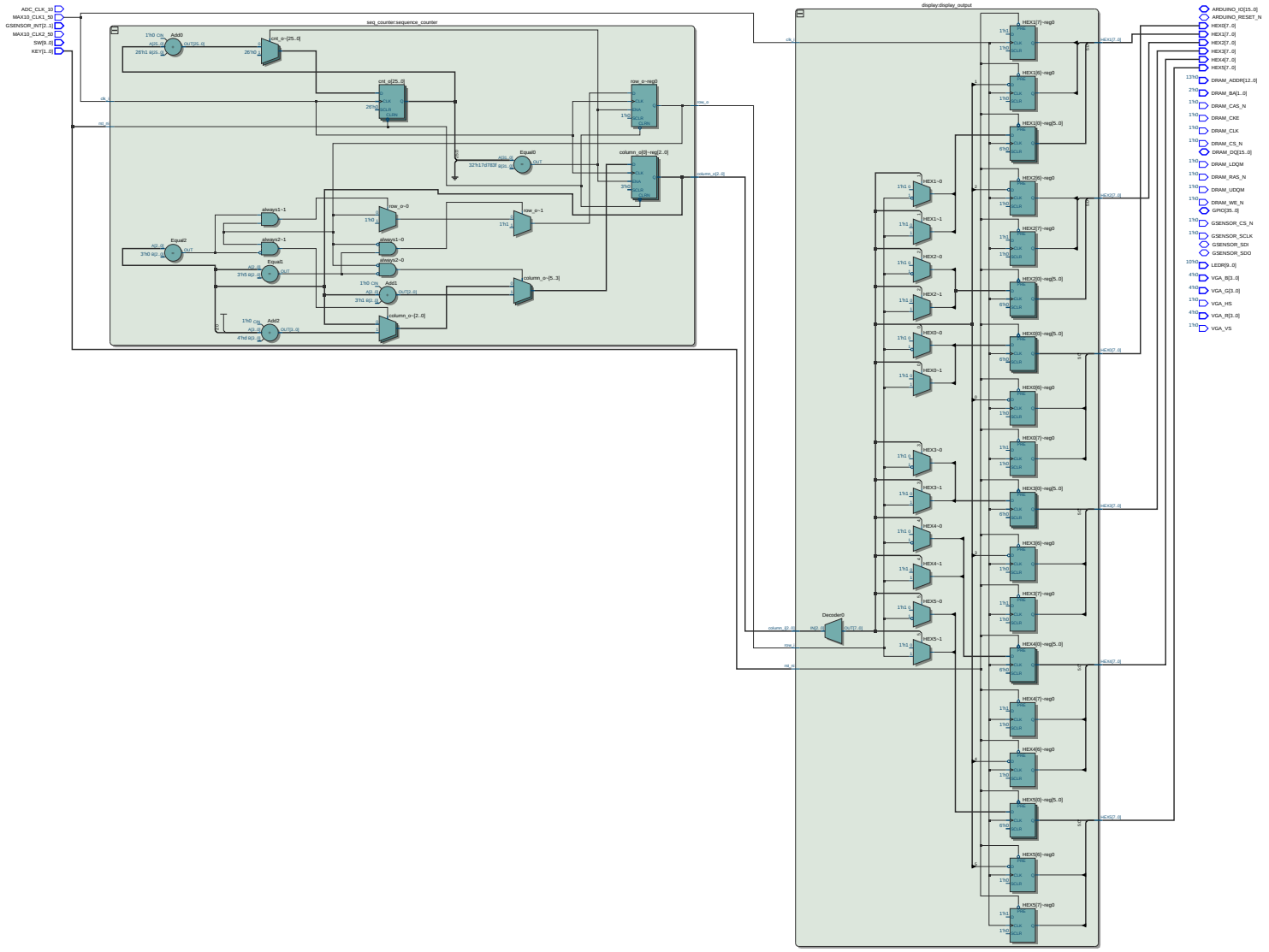

ADC_CLK_10
MAX10_CLK1_50
GSENSOR_INT[2..1]
MAX10_CLK2_50
SW[9..0]
KEY[1..0]
seq_counter:sequence_counter
clk_i
rst_ni
column_o[2..0]
row_o
display:display_output
clk_i
column_i[2..0]
row_i
rst_ni
HEX0[7..0]
HEX1[7..0]
HEX2[7..0]
HEX3[7..0]
HEX4[7..0]
HEX5[7..0]
ARDUINO_IO[15..0]
ARDUINO_RESET_N
HEX0[7..0]
HEX1[7..0]
HEX2[7..0]
HEX3[7..0]
HEX4[7..0]
HEX5[7..0]
13'h0
DRAM_ADDR[12..0]
2'h0
DRAM_BA[1..0]
1'h0
DRAM_CAS_N
1'h0
DRAM_CKE
1'h0
DRAM_CLK
1'h0
DRAM_CS_N
DRAM_DQ[15..0]
1'h0
DRAM_LDQM
1'h0
DRAM_RAS_N
1'h0
DRAM_UDQM
1'h0
DRAM_WE_N
GPIO[35..0]
1'h0
GSENSOR_CS_N
1'h0
GSENSOR_SCLK
GSENSOR_SDI
GSENSOR_SDO
10'h0
LEDR[9..0]
4'h0
VGA_B[3..0]
4'h0
VGA_G[3..0]
1'h0
VGA_HS
4'h0
VGA_R[3..0]
1'h0
VGA_VS


seq_counter:sequence_counter
clk_i
rst_ni
Add0
1'h0 CIN
A[25..0]
26'h1 B[25..0]
OUT[25..0]
cnt_o~[25..0]
26'h0
cnt_o[25..0]
D
CLK
Q
SCLR
CLRN
row_o~reg0
ENA
1'h0
row_o
Equal0
A[31..0]
32'h17d783f B[31..0]
OUT
25:0
column_o[0]~reg[2..0]
3'h0
column_o[2..0]
always1~1
always2~1
always1~0
always2~0
row_o~0
row_o~1
1'h1
Equal2
3'h0 B[2..0]
Equal1
3'h5 B[2..0]
Add1
3'h1 B[2..0]
column_o~[5..3]
column_o~[2..0]
Add2
4'hd B[3..0]
OUT[3..0]
2:0


display:display_output
clk_i
column_i[2..0]
row_i
rst_ni
Decoder0
IN[2..0]
OUT[7..0]
HEX1[7]~reg0
HEX1[6]~reg0
HEX1[0]~reg[5..0]
HEX1[7..0]
HEX2[6]~reg0
HEX2[7]~reg0
HEX2[0]~reg[5..0]
HEX2[7..0]
HEX0[0]~reg[5..0]
HEX0[6]~reg0
HEX0[7]~reg0
HEX0[7..0]
HEX3[0]~reg[5..0]
HEX3[6]~reg0
HEX3[7]~reg0
HEX3[7..0]
HEX4[0]~reg[5..0]
HEX4[7]~reg0
HEX4[6]~reg0
HEX4[7..0]
HEX5[0]~reg[5..0]
HEX5[6]~reg0
HEX5[7]~reg0
HEX5[7..0]
HEX1~0
HEX1~1
HEX2~0
HEX2~1
HEX0~0
HEX0~1
HEX3~0
HEX3~1
HEX4~0
HEX4~1
HEX5~0
HEX5~1
PRE
D
CLK
Q
SCLR
1'h1
1'h0
6'h0


ADC_CLK_10
MAX10_CLK1_50
GSENSOR_INT[2..1]
MAX10_CLK2_50
SW[9..0]
KEY[1..0]
seq_counter:sequence_counter
Add0
cnt_o~[25..0]
cnt_o[25..0]
row_o~reg0
Equal0
column_o[2]~reg[2..0]
always1~1
row_o~0
row_o~1
Equal2
always2~1
always1~0
always2~0
Equal1
Add1
column_o~[5..3]
Add2
column_o~[2..0]
display:display_output
HEX1[7]~reg0
HEX1[6]~reg0
HEX1[0]~reg[5..0]
HEX2[6]~reg0
HEX2[7]~reg0
HEX2[0]~reg[5..0]
HEX0[6]~reg0
HEX0[7]~reg0
HEX0[0]~reg[5..0]
HEX3[6]~reg0
HEX3[7]~reg0
HEX3[0]~reg[5..0]
HEX4[6]~reg0
HEX4[7]~reg0
HEX4[0]~reg[5..0]
HEX5[6]~reg0
HEX5[7]~reg0
HEX1-0
HEX1-1
HEX2-0
HEX2-1
HEX0-0
HEX0-1
HEX3-0
HEX3-1
HEX4-0
HEX4-1
HEX5-0
HEX5-1
Decoder0
ARDUINO_IO[15..0]
ARDUINO_RESET_N
HEX0[7..0]
HEX1[7..0]
HEX2[7..0]
HEX3[7..0]
HEX4[7..0]
HEX5[7..0]
DRAM_ADDR[12..0]
DRAM_BA[1..0]
DRAM_CAS_N
DRAM_CKE
DRAM_CLK
DRAM_CS_N
DRAM_DQ[15..0]
DRAM_LDQM
DRAM_RAS_N
DRAM_UDQM
DRAM_WE_N
GPIO[35..0]
GSENSOR_CS_N
GSENSOR_SCLK
GSENSOR_SDI
GSENSOR_SDO
LEDR[9..0]
VGA_B[3..0]
VGA_G[3..0]
VGA_HS
VGA_R[3..0]
VGA_VS

```
// ============================================================================
//   Ver  :| Author              :| Mod. Date :| Changes Made:
//   V1.1 :| Alexandra Du        :| 06/01/2016:| Added Verilog file
// ============================================================================


//=======================================================
//  This code is generated by Terasic System Builder
//=======================================================

`define ENABLE_ADC_CLOCK
`define ENABLE_CLOCK1
`define ENABLE_CLOCK2
`define ENABLE_SDRAM
`define ENABLE_HEX0
`define ENABLE_HEX1
`define ENABLE_HEX2
`define ENABLE_HEX3
`define ENABLE_HEX4
`define ENABLE_HEX5
`define ENABLE_KEY
`define ENABLE_LED
`define ENABLE_SW
`define ENABLE_VGA
`define ENABLE_ACCELEROMETER
`define ENABLE_ARDUINO
`define ENABLE_GPIO

module DE10_LITE_Golden_Top(

   ////////////// ADC CLOCK: 3.3-V LVTTL //////////
`ifdef ENABLE_ADC_CLOCK
   input                       ADC_CLK_10,
`endif
   ////////////// CLOCK 1: 3.3-V LVTTL //////////
`ifdef ENABLE_CLOCK1
   input                       MAX10_CLK1_50,
`endif
   ////////////// CLOCK 2: 3.3-V LVTTL //////////
`ifdef ENABLE_CLOCK2
   input                       MAX10_CLK2_50,
`endif

   ////////////// SDRAM: 3.3-V LVTTL //////////
`ifdef ENABLE_SDRAM
   output          [12:0]      DRAM_ADDR,
   output           [1:0]      DRAM_BA,
   output                      DRAM_CAS_N,
   output                      DRAM_CKE,
   output                      DRAM_CLK,
   output                      DRAM_CS_N,
   inout           [15:0]      DRAM_DQ,
   output                      DRAM_LDQM,
   output                      DRAM_RAS_N,
   output                      DRAM_UDQM,
   output                      DRAM_WE_N,
`endif
```

```
58
59      ////////////// SEG7: 3.3-V LVTTL //////////
60    `ifdef ENABLE_HEX0
61      output           [7:0]     HEX0,
62    `endif
63    `ifdef ENABLE_HEX1
64      output           [7:0]     HEX1,
65    `endif
66    `ifdef ENABLE_HEX2
67      output           [7:0]     HEX2,
68    `endif
69    `ifdef ENABLE_HEX3
70      output           [7:0]     HEX3,
71    `endif
72    `ifdef ENABLE_HEX4
73      output           [7:0]     HEX4,
74    `endif
75    `ifdef ENABLE_HEX5
76      output           [7:0]     HEX5,
77    `endif
78
79      ////////////// KEY: 3.3 V SCHMITT TRIGGER //////////
80    `ifdef ENABLE_KEY
81      input            [1:0]     KEY,
82    `endif
83
84      ////////////// LED: 3.3-V LVTTL //////////
85    `ifdef ENABLE_LED
86      output           [9:0]     LEDR,
87    `endif
88
89      ////////////// SW: 3.3-V LVTTL //////////
90    `ifdef ENABLE_SW
91      input            [9:0]     SW,
92    `endif
93
94      ////////////// VGA: 3.3-V LVTTL //////////
95    `ifdef ENABLE_VGA
96      output           [3:0]     VGA_B,
97      output           [3:0]     VGA_G,
98      output                     VGA_HS,
99      output           [3:0]     VGA_R,
100     output                     VGA_VS,
101   `endif
102
103     ////////////// Accelerometer: 3.3-V LVTTL //////////
104   `ifdef ENABLE_ACCELEROMETER
105     output                     GSENSOR_CS_N,
106     input            [2:1]     GSENSOR_INT,
107     output                     GSENSOR_SCLK,
108     inout                      GSENSOR_SDI,
109     inout                      GSENSOR_SDO,
110   `endif
111
112     ////////////// Arduino: 3.3-V LVTTL //////////
113   `ifdef ENABLE_ARDUINO
114     inout           [15:0]     ARDUINO_IO,
```

```
115     inout                       ARDUINO_RESET_N,
116   `endif
117
118     ////////////// GPIO, GPIO connect to GPIO Default: 3.3-V LVTTL //////////
119   `ifdef ENABLE_GPIO
120     inout           [35:0]      GPIO
121   `endif
122   );
123
124
125   wire [2:0]          column;
126   wire                row;
127
128   seq_counter #(
129       .CLK_FREQ       (50000000),
130       .UPDATE_HZ      (2)
131   ) sequence_counter (
132       .clk_i          (MAX10_CLK1_50),
133       .rst_ni         (KEY[0]),
134       .column_o       (column),
135       .row_o          (row)
136   );
137
138   display display_output (
139       .clk_i      (MAX10_CLK1_50),
140       .rst_ni     (KEY[0]),
141       .column_i   (column),
142       .row_i      (row),
143       .HEX0       (HEX0),
144       .HEX1       (HEX1),
145       .HEX2       (HEX2),
146       .HEX3       (HEX3),
147       .HEX4       (HEX4),
148       .HEX5       (HEX5)
149   );
150   endmodule
151
```

```
1   module display (
2       input               clk_i,
3       input               rst_ni,
4       input [2:0]         column_i ,
5       input               row_i,
6       output reg [7:0]     HEX0,
7       output reg [7:0]     HEX1,
8       output reg [7:0]     HEX2,
9       output reg [7:0]     HEX3,
10      output reg [7:0]     HEX4,
11      output reg [7:0]     HEX5
12
13  );
14
15  localparam upper_mask = 8'b1001_1100;
16  localparam lower_mask = 8'b1010_0011;
17
18  always @(posedge clk_i or negedge rst_ni) begin
19      if (!rst_ni) begin
20          HEX0 <= 8'hFF;
21          HEX1 <= 8'hFF;
22          HEX2 <= 8'hFF;
23          HEX3 <= 8'hFF;
24          HEX4 <= 8'hFF;
25          HEX5 <= 8'hFF;
26      end else begin
27
28          // Turn off all displays mask (common anode)
29          HEX0 <= 8'hFF;
30          HEX1 <= 8'hFF;
31          HEX2 <= 8'hFF;
32          HEX3 <= 8'hFF;
33          HEX4 <= 8'hFF;
34          HEX5 <= 8'hFF;
35
36          // Overwrite only selected display
37          case (column_i)
38              3'd0: HEX0 <= row_i ? upper_mask : lower_mask;
39              3'd1: HEX1 <= row_i ? upper_mask : lower_mask;
40              3'd2: HEX2 <= row_i ? upper_mask : lower_mask;
41              3'd3: HEX3 <= row_i ? upper_mask : lower_mask;
42              3'd4: HEX4 <= row_i ? upper_mask : lower_mask;
43              3'd5: HEX5 <= row_i ? upper_mask : lower_mask;
44          endcase
45      end
46  end
47
48  endmodule
```

```
1   module seq_counter #(
2       parameter CLK_FREQ      = 50_000_000,
3       parameter UPDATE_HZ     = 2,
4       parameter DISP_WIDTH    = 10
5   )(
6       input               clk_i,
7       input               rst_ni,
8
9       output reg [2:0]    column_o,
10      output reg          row_o
11  );
12
13  localparam CNT_STOP = CLK_FREQ / UPDATE_HZ;
14  localparam WIDTH = $clog2(CLK_FREQ);
15
16  reg [WIDTH-1:0] cnt_o;
17  wire            tick_w = (cnt_o == CNT_STOP - 1);
18
19  always @(posedge clk_i or negedge rst_ni)
20      if (!rst_ni)     cnt_o <= 0;
21      else if (tick_w) cnt_o <= 0;
22      else             cnt_o <= cnt_o + 1;
23
24
25  always @(posedge clk_i or negedge rst_ni)
26      if            (!rst_ni)                                 row_o <= 1'b0;
27      else if       (tick_w)
28          if        (row_o == 1'b0 && column_o == 3'b101)  row_o <= 1'b1;
29          else if   (row_o == 1'b1 && column_o == 3'b000)  row_o <= 1'b0;
30
31
32  always @(posedge clk_i or negedge rst_ni)
33      if            (!rst_ni)                                 column_o <= 3'd0;
34      else if       (tick_w)
35          if        (row_o == 1'b0 && column_o != 3'b101)  column_o <= column_o + 1'b1;
36          else if   (row_o == 1'b1 && column_o != 3'b000)  column_o <= column_o - 1'b1;
37
38  endmodule
```